



Rapport final du Projet Smart Data Grids

Membres du groupe :

Fellati Zineb
Mecherak Thanina
Cissé Mamadou
Mankai Latifa

Lien Github:

<https://github.com/Ciscom224/smart-grid-app/tree/main>

M2 Datascale
Année universitaire : 2025-2026

Table de matières

Vue d'ensemble du projet.....	3
Contexte et motivation.....	3
1. Formule de calcul des profils profils horaires de consommation:.....	4
Définition du coefficient horaire:.....	4
Justification de la méthode:.....	4
2. Calcul des potentiels de production:.....	5
3. Prétraitement des données :.....	5
a) Source RTE (Profil de Consommation).....	5
b) Source: Bâtiments.....	6
c) Source : Réseau électrique souterrain.....	7
d) Source: Administration.....	7
e) Source : Potentiel solaire.....	7
f) Connection des bâtiments au réseau.....	7
4. Modélisation du graphe :.....	8
5. Schéma Neo4j :.....	10
6. Critères de clustering.....	11
8. Requêtes CYPHER :.....	13
9. Implémentation de l'application.....	13
10. Stack utilisé.....	17
Conclusion.....	18



Vue d'ensemble du projet

Contexte et motivation

La gestion des systèmes énergétiques connaît une évolution majeure avec l'essor des réseaux électriques décentralisés, également appelés *smart grids*. Contrairement aux réseaux traditionnels, principalement conçus pour une production centralisée et une distribution unidirectionnelle de l'électricité, ces nouveaux systèmes intègrent une diversité d'acteurs et de flux énergétiques.

On distingue notamment :

- les consommateurs, qui utilisent l'électricité pour leurs besoins résidentiels ou industriels;
- les producteurs, qui génèrent de l'énergie localement, par exemple à partir de panneaux photovoltaïques.
- les prosumers, qui combinent les deux rôles en produisant et en consommant de l'électricité.

Cette évolution permet le développement de l'autoconsommation locale et collective, dans laquelle l'énergie produite au sein d'un territoire (quartier, commune, communauté) est partagée entre ses différents acteurs.

Les principaux objectifs poursuivis sont :

1. la réduction des coûts énergétiques pour les participants ;
2. la diminution de l'empreinte carbone associée à la production et au transport de l'électricité ;
3. la promotion de modèles d'autoconsommation collective plus efficaces et équitables.

1. Formule de calcul des profils horaires de consommation:

Afin d'estimer la consommation électrique horaire de chaque adresse à partir de données annuelles agrégées, une méthode de répartition proportionnelle basée sur le profil de consommation national a été retenue.

La consommation horaire d'une adresse aaa à l'heure h est calculée à la volée selon la formule suivante :

$$\text{Conso}_{a,h} = \text{Conso}_{a,ann} \times \text{coef}_h$$

où :

- $\text{Conso}_{a,h}$ est la consommation de l'adresse a à l'heure h (en MWh),
- $\text{Conso}_{a,ann}$ est la consommation annuelle totale de l'adresse a (en MWh),
- coef_h est le coefficient horaire de répartition.

Définition du coefficient horaire:

Le coefficient horaire coef_h représente la part de la consommation annuelle nationale observée à l'heure h . Il est défini comme suit :

$$\text{coef}_h = \frac{\text{Consommation France à l'heure } h}{\text{Consommation France annuelle}}$$

Ce coefficient est calculé à partir des données de consommation électrique nationale publiées par RTE, agrégées à l'échelle horaire.

Justification de la méthode:

Cette approche permet de :

- préserver le volume annuel de consommation propre à chaque adresse,
- reproduire fidèlement les variations temporelles réelles de la demande électrique (cycles journaliers, hebdomadaires et saisonniers),
- éviter l'introduction d'hypothèses arbitraires sur les profils de consommation.

La consommation horaire n'est pas stockée mais calculée dynamiquement à partir de la consommation annuelle et des coefficients horaires, ce qui permet de limiter les volumes de données tout en conservant un profil temporel réaliste.

2. Calcul des potentiels de production:

Pour estimer la production solaire journalière, nous nous sommes appuyés sur les données historiques de production solaire de l'année 2023. Dans un premier temps, la production solaire journalière a été obtenue en agrégeant les valeurs horaires de production pour chaque jour, c'est-à-dire en calculant la somme des productions enregistrées sur les 24 heures de la journée. Cette étape permet d'obtenir une série temporelle journalière représentant l'évolution réelle de la production solaire.

Ensuite, afin de répartir la production solaire dans le temps, nous avons calculé un **coefficient journalier** représentant la part de la production d'un jour par rapport à la production totale du mois auquel il appartient. Ce coefficient est défini par la formule suivante :

$$coef_{journalier}(j) = \frac{production_{journaliere}(j)}{production_{mensuelle}(m)}$$

où **productionjournalière(j)** correspond à la production solaire du jour jjj, et **productionmensuelle(m)** représente la production solaire totale du mois mmm contenant ce jour.

Enfin, la production solaire journalière d'un bâtiment est estimée en fonction de sa consommation annuelle et du coefficient journalier selon la relation suivante :

$$Production_{jour} = \alpha \times Consommation_{annuelle} \times coef_{journalier}$$

où α est un coefficient de conversion fixé à 0,8. Cette approche permet de générer des profils de production solaire journaliers réalistes tout en conservant les variations saisonnières observées dans les données historiques.

3. Prétraitement des données :

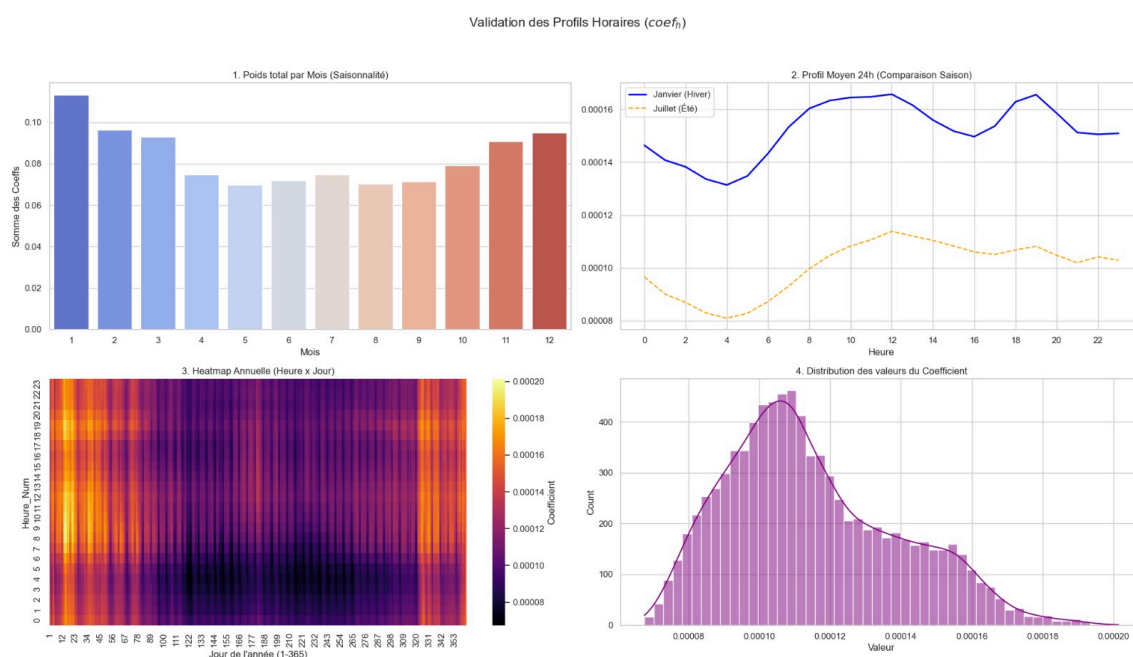
a) Source RTE (Profil de Consommation)

➤ Fichier: "profil_synthetiques.csv"

Pour les données de consommation horaire fournies par RTE, nous avons traité les valeurs manquantes par interpolation afin de ne pas créer des trous dans les séries temporelles. Nous avons aussi traité les doublons en gardant seulement la première occurrence pour chaque date et heure.

Nous avons, par la suite, procédé à une analyse graphique multidimensionnelle détaillée.

- **Graphique 1 (Poids par Mois)** : Un diagramme en barres pour comparer la consommation en hiver (bleu) et en été (rouge).
- **Graphique 2 (Profil journalier)** : ici on compare la consommation moyenne pour chaque heure sur toutes les journées du mois de janvier(hiver) et du mois de juillet(été). On observe que les pics de consommation le matin et le soir sont plus marqués en hiver.
- **Graphique 3 (Heatmap)** : Une carte de chaleur permettant de visualiser simultanément la **saisonnalité annuelle** et le **cycle journalier** de la consommation. En parcourant le graphique de gauche à droite, on observe les variations de consommation au fil des mois : les zones claires correspondent à des périodes de forte consommation (hiver) et les zones sombres à des périodes plus faibles (été). En lisant verticalement chaque colonne, on peut suivre l'évolution de la consommation au cours d'une journée : les heures claires indiquent les pics de consommation, généralement le matin et le soir, tandis que les heures sombres représentent les creux, notamment en pleine nuit.
- **Graphique 4 (Distribution)** : Un histogramme pour voir la répartition des valeurs des coefficients.



b) Source: Bâtiments

➤ Fichier: “consommation-annuelle-residentielle-par-adresse.csv”

Pour les données des bâtiments résidentiels d’Île-de-France, nous avons d’abord sélectionné et renommé les colonnes pertinentes : code IRIS (code_iris), code commune (code_commune), adresse (adresse), nombre de logements (nb_logements) et consommation annuelle en MWh (conso). Nous avons supprimé les adresses sans consommation ou sans

information de localisation. En cas de doublons pour une même adresse, nous avons additionné les consommations afin d'obtenir la consommation totale du bâtiment. Nous avons également créé un fichier référentiel unique (iris_file.csv) pour les IRIS après dédoublement, et généré un identifiant unique (UID) pour chaque bâtiment à partir du code commune et de l'adresse. Enfin, pour produire les courbes de charge individuelles, nous avons réalisé un produit cartésien entre la table des bâtiments et les profils RTE, ce qui nous a permis de calculer la consommation journalière ($\text{conso_jour} = \text{conso_annuelle} \times \text{coefficient_journalier}$). Nous avons ensuite converti les séries temporelles ainsi obtenues au format JSON afin de les stocker directement dans Neo4j.

c) Source : Réseau électrique souterrain

➤ Fichier: “réseau-souterrain-basse-tension-bt-2.csv”

L'objectif de prétraitement de cette source est de préparer les données géométriques et administratives du réseau basse tension pour le graphe. Pour cela, nous avons conservé les colonnes essentielles (geometry, nom_grd, date_maj, code_epci). Nous avons supprimé les lignes sans code EPCI et éliminé les doublons pour éviter les répétitions. Enfin, nous avons créé un identifiant unique (id_segment) pour chaque segment de réseau.

d) Source: Administration

➤ Fichier: “consommation-electrique-par-secteur-dactivite-iris.csv”

Cette source fournit les données administratives et IRIS, utilisées pour construire la hiérarchie (Région → Département → EPCI → Commune → IRIS) dans le graphe. Nous avons corrigé, en premier lieu, l'encodage, sélectionné et renommé les colonnes importantes (codes et noms des entités, catégories et consommation moyenne), nettoyé les codes et converti les consommations en valeurs numériques, et créé des référentiels uniques pour chaque entité.

e) Source : Potentiel solaire

➤ Fichier: “building_gps.csv”

Notre objectif c'est de simuler la production d'électricité solaire pour chaque bâtiment. Comme on ne dispose pas des panneaux réels, on crée un modèle dans lequel:

- On décide aléatoirement si le bâtiment a un panneau (30 % de chance).
- Si oui, on calcule sa puissance installée selon le nombre de logements.
- On applique le modèle solaire (cycle saisonnier + météo) pour obtenir la production journalière et annuelle.
- On stocke la production journalière en JSON pour chaque date.
- Les bâtiments sans panneaux ont une production nulle.

f) Connection des bâtiments au réseau

Dans cette étape, nous relierons chaque bâtiment au segment de réseau souterrain le plus proche dans Neo4j en créant des relations CONNECTED qui indiquent la distance en mètres. Pour cela, nous utilisons le fichier distances_calculees.csv, contenant pour chaque bâtiment l'adresse, l'ID du segment, le code IRIS et la distance précalculée. Cette approche nous permet de connecter les bâtiments au réseau de manière précise et fiable.

4. Modélisation du graphe :

a) Noeuds

Intitulé du noeud	Attributs	Description
Région	numero_region,nom_region	Représente une division administrative de niveau régional, utilisée pour regrouper les départements et analyser les données énergétiques à une échelle macro-territoriale.
Departement	numero_dept, nom_dept	Subdivision administrative d'une région. Permet d'affiner l'analyse territoriale des consommations et productions énergétiques.
Commune	code_commune,nom_commune	Unité administrative locale correspondant à une ville ou un village. Sert à localiser précisément les bâtiments et les données de consommation
Epsi	code_epci,nom_epci	Établissement Public de Coopération Intercommunale. Regroupe plusieurs communes afin de mutualiser la gestion territoriale et énergétique.
Iris	nom_iris,code_iris, type_iris	Zone statistique
Building	adresse,nombres logement, consommation annuelle,	Bâtiment / logement avec sa consommation énergétique,

	Date, Label (producer,consumer)	pouvant être associée à un consommateur ou à un producteur d'énergie.
Profil	Date, heure, valeur,adresse	Profil de consommation temporelle (horaire)
GridSegment	geolocalisation, code_iris	Segment du réseau électrique ou énergétique
Potentiel	id_potentiel,date,adresse,production	Représente la distance géographique entre un bâtiment (Building) et les segments du réseau électrique ou énergétique (GridSegment) auxquels il peut être rattaché.

b) Relations

Source	Relation	Cible
Departement	located_IN	Region
commune	located_IN	Departement
commune	located_IN	Epsi
Iris	located_IN	commune
Building	belongs_TO	Iris
Building	connected	GridSegement
Building	has_profil	profil
Building	has_potentiel	potentiel
Consumer	belongs_TO	Iris
Consumer	has_profil	profil
Consumer	has_potentiel	Potentiel

Consumer	connected	GridSegement
prosumer	belongs_TO	Iris
Prosumer	has_profil	profil
Prosumer	has_potentiel	Potentiel
Prosumer	connected	GridSegement

5. Schéma Neo4j :

Nous représentons ici le graphe du système énergétique à l'échelle territoriale, intégrant les dimensions administratives, énergétiques et réseau.

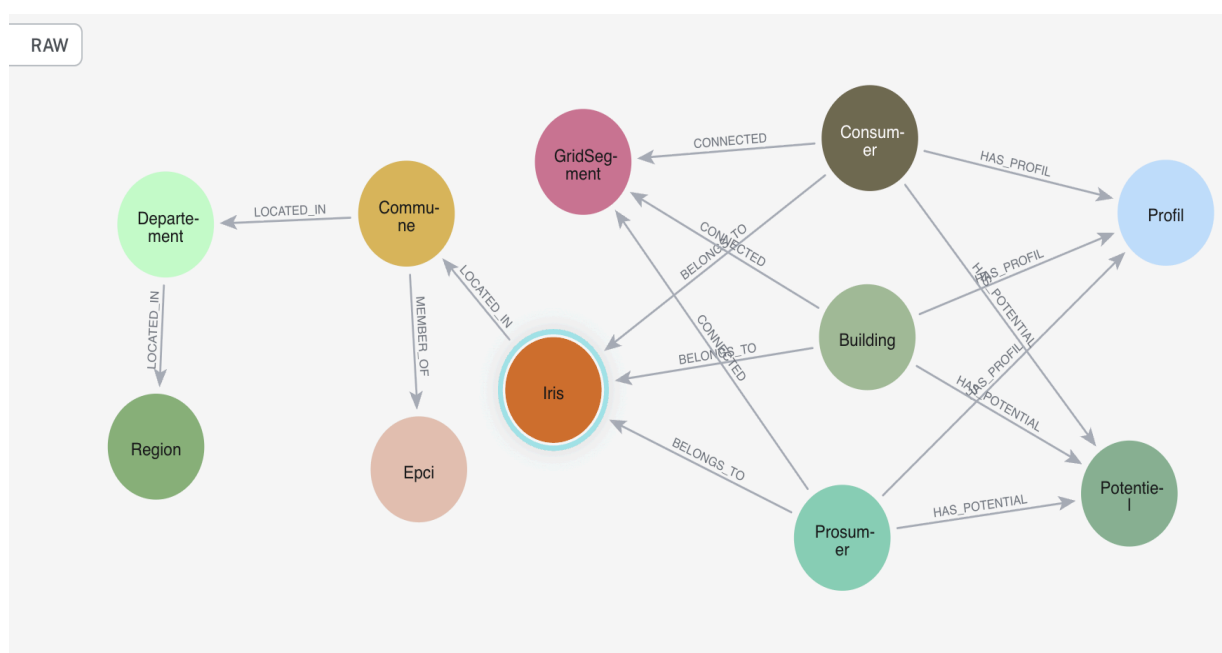
Il organise d'abord le territoire selon une **hiérarchie administrative** :

Région → **Département** → **Commune** → **IRIS**, ce qui permet de localiser précisément les données.

Les **bâtiments** et les **acteurs énergétiques** (Consumer et Prosumer) sont rattachés à un **IRIS** et connectés à un **segment du réseau énergétique (GridSegment)**.

Chaque consommateur ou prosumer possède un **profil temporel** décrivant l'évolution horaire de la consommation ou de la production.

Enfin, nous intégrons dans le graphe le **potentiel énergétique** des bâtiments permettant d'analyser les capacités de production locale.



6. Critères de clustering

Afin d'analyser efficacement le réseau électrique intelligent, nous avons défini plusieurs critères de clustering dans l'interface de l'application. Ces critères nous permettent de filtrer les bâtiments et de former des groupes cohérents en fonction de leur position géographique et de leurs caractéristiques énergétiques.

- **Distance de liaison au Grid**

Ce critère nous permet de sélectionner les bâtiments en fonction de leur distance par rapport au réseau électrique. Nous définissons une distance minimale et maximale afin d'inclure uniquement les bâtiments situés à proximité du segment de réseau étudié. Cela nous permet de créer des clusters réalistes, car les bâtiments regroupés sont physiquement connectés ou proches de la même infrastructure électrique.

- **Rayon d'analyse**

Le rayon d'analyse correspond à la zone géographique autour du point de départ choisi par l'utilisateur. Concrètement, lorsque nous saisissons une adresse ou une localisation, l'application recherche tous les bâtiments situés dans ce périmètre. En ajustant ce rayon, nous pouvons étudier soit un quartier précis, soit une zone plus large.

- **Nombre minimum de bâtiments**

Avec ce paramètre, nous définissons le nombre minimal de bâtiments nécessaires pour former le cluster.

- **Consommation minimale (MWh/an)**

Ce critère nous permet de filtrer les bâtiments selon leur consommation énergétique annuelle. Nous pouvons choisir de ne prendre en compte que les bâtiments ayant une consommation supérieure à un certain seuil. Cette approche nous aide à concentrer notre analyse sur les bâtiments ayant un impact énergétique important dans le réseau.

- **Taux minimum d'autosuffisance (%)**

Le taux d'autosuffisance indique combien d'énergie un bâtiment peut produire lui-même par rapport à ce qu'il consomme. Ce paramètre nous permet d'identifier les bâtiments capables de produire une partie de leur propre énergie. En fixant un seuil minimum d'autosuffisance, nous sélectionnons uniquement les bâtiments dont la production énergétique couvre une certaine proportion de leur consommation.

- **Afficher seulement les surplus**

Cette option permet d'afficher uniquement les bâtiments qui produisent plus d'énergie qu'ils n'en consomment. Cette recherche est utile car l'énergie produite en excès (surplus), peut être injectée dans le réseau électrique et utilisée par d'autres bâtiments.

7. Critères d'évaluation de la validité d'une configuration

	Critère	Description
C1	Taux d'autosuffisance	Une grille est considérée autosuffisante lorsque la somme des productions d'énergie de la grille sur une période donnée est supérieure ou égale à la somme des consommations de l'ensemble des bâtiments connectés à cette même grille sur la même période.
C2	Proximité aux GridSegments	Chaque bâtiment doit être raccordable à au moins un GridSegment situé à une distance inférieure ou égale à un seuil défini

Une configuration est considérée valide si et seulement si les critères C1 et C2 sont simultanément respectés.

8. Requêtes CYPHER :

Nous avons mis en place deux types de requêtes dans Neo4j pour analyser les bâtiments et leur connexion au réseau :

a) Agrégation par segment de réseau (Cluster global)

La première requête identifie tous les bâtiments connectés à chaque segment de réseau et calcule pour chaque cluster : le nombre de bâtiments, la consommation totale, la production solaire totale et le ratio d'autosuffisance. Des filtres sont appliqués pour ne conserver que les clusters significatifs, comportant au moins un consommateur pur et atteignant un seuil minimum d'autosuffisance. Le résultat est structuré sous forme de JSON regroupant les KPIs globaux et la liste des bâtiments pour chaque cluster, permettant une visualisation et une analyse globale du territoire.

b) Analyse locale autour d'un bâtiment centre (Vue Focus)

La deuxième requête permet de se concentrer sur un bâtiment cible et sur tous les bâtiments situés dans un rayon défini autour de celui-ci. Pour chaque bâtiment du cluster local, la consommation, la production solaire, l'autosuffisance individuelle et les distances au réseau sont récupérées. Des filtres sont appliqués pour garantir la présence d'au moins un consommateur pur et un ratio d'autosuffisance minimal pour le groupe. Le résultat final est

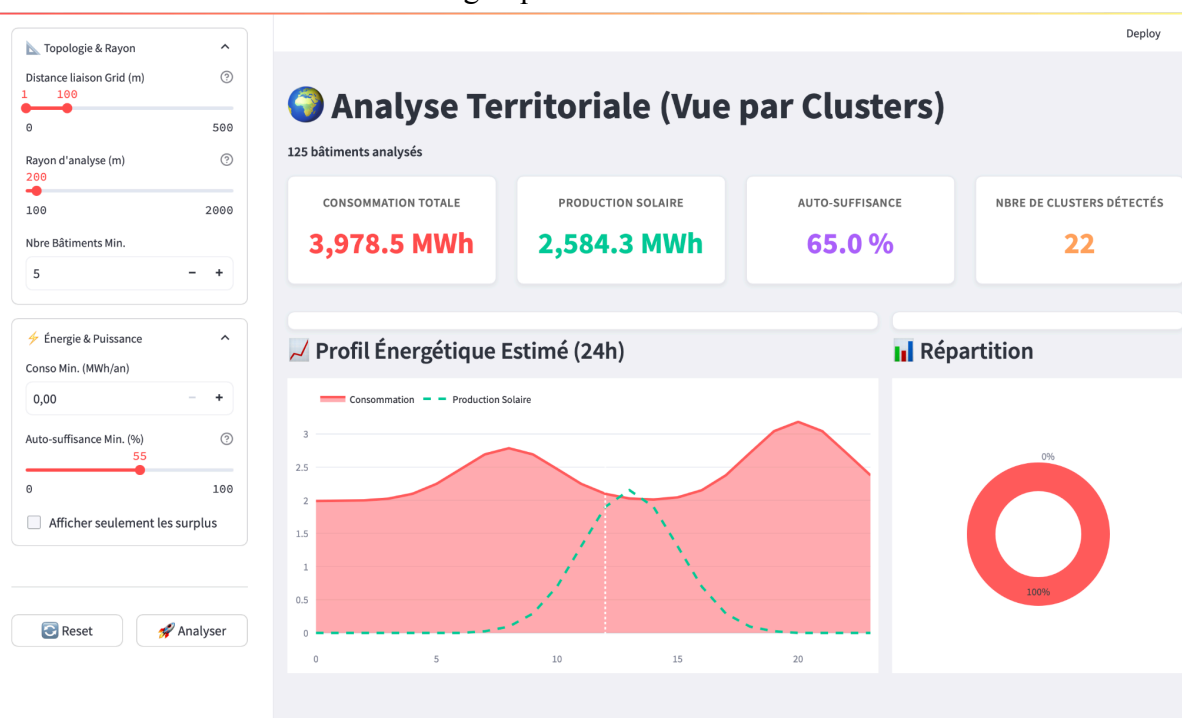
un JSON détaillé contenant les KPIs du cluster local, la liste complète des bâtiments avec leurs courbes de charge et de production, ainsi que les coordonnées du centre.

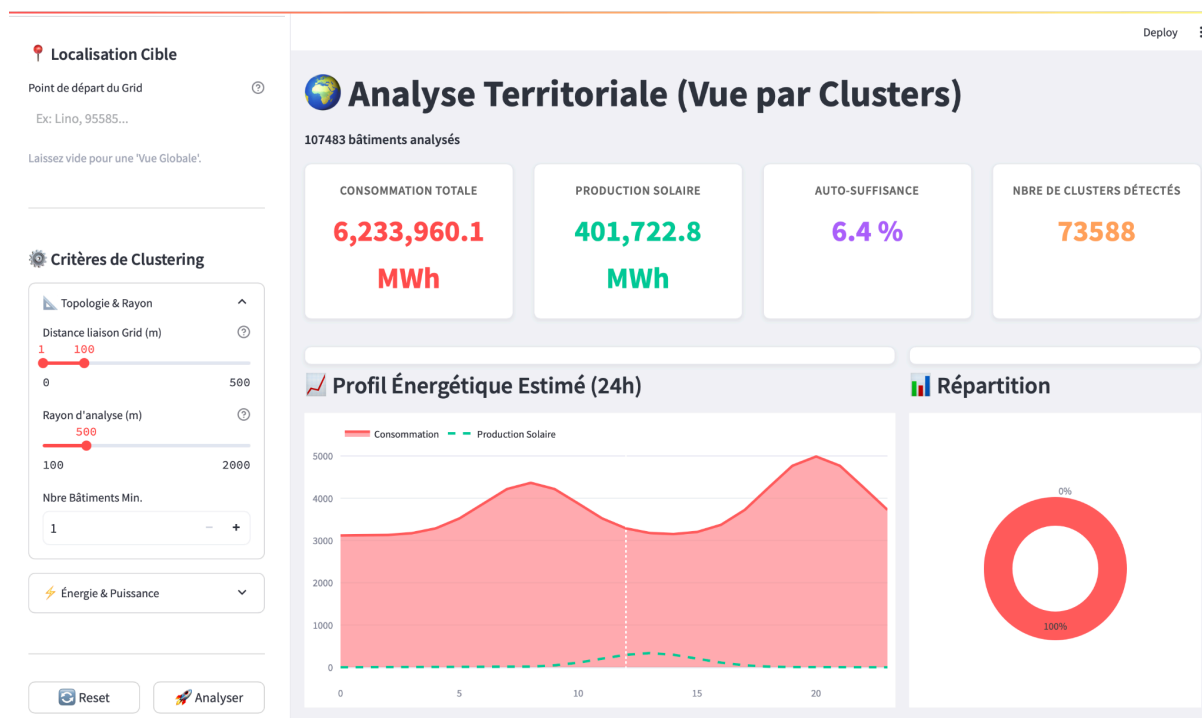
9. Implémentation de l'application

Nous fournissons dans chaque partie une description de l'interface web **SMART-GRID-APP** qui permet de visualiser et d'analyser les données énergétiques d'un cluster de bâtiments à partir d'un point géographique donné. Comme illustré dans la figure, l'utilisateur peut définir une localisation cible ainsi que plusieurs paramètres de clustering, tels que la distance de liaison au réseau, le rayon d'analyse et le nombre minimum de bâtiments. L'application affiche ensuite différents indicateurs clés, notamment la consommation totale d'énergie, la production solaire, le taux d'autosuffisance énergétique et le nombre de bâtiments prosumers. Elle propose également des visualisations interactives, telles qu'un profil énergétique estimé sur 24 heures comparant la consommation et la production solaire, ainsi qu'un graphique de répartition permettant d'analyser la distribution énergétique au sein du cluster. Cette interface facilite ainsi l'exploration et l'interprétation des dynamiques énergétiques locales afin d'optimiser la gestion des réseaux électriques intelligents.

a) Analyse Territoriale (Globale) :

Analyse par clusters IRIS et segments de réseau Enedis, avec encerclement visuel (halos) pour identifier les communautés énergétiques.



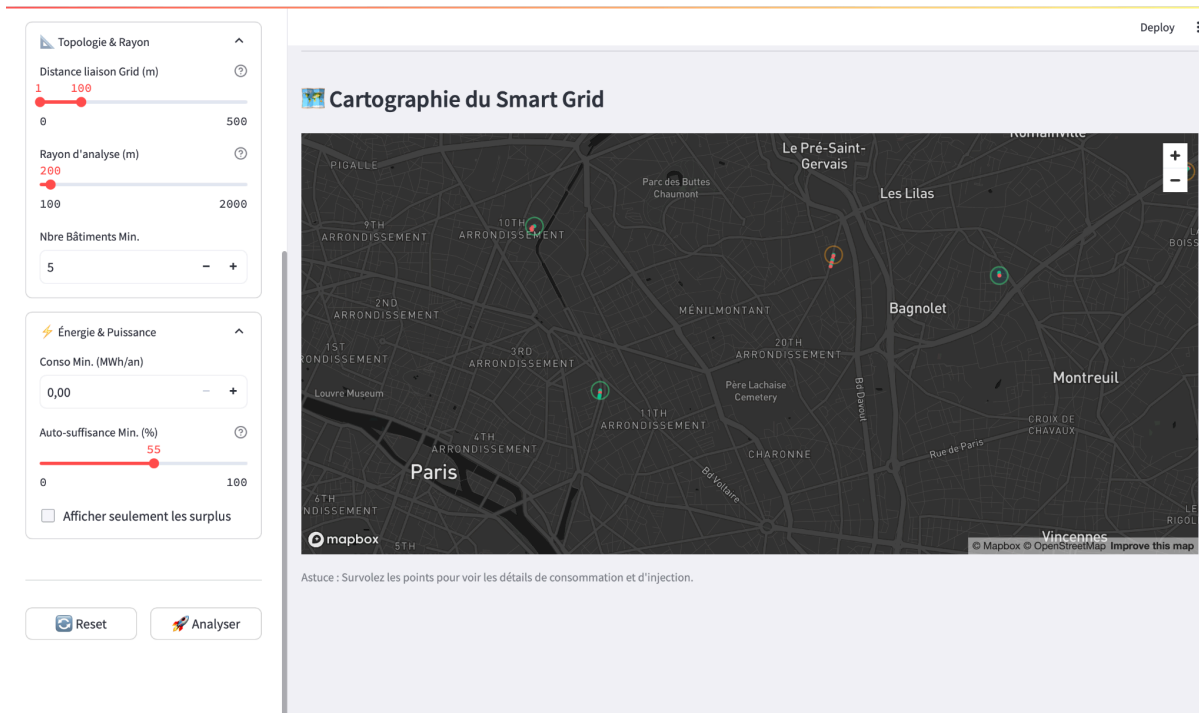


L'interface est constituée aussi d'une carte interactive permettant de visualiser la répartition des bâtiments et leur comportement énergétique dans la zone étudiée.

Dans la figure ci-dessous, plusieurs points géographiques sont affichés pour représenter des bâtiments ou des regroupements de bâtiments selon les critères choisis sur le panneau de gauche comme la distance liaison Grid(m), le rayon d'analyse(m), le taux d'auto-suffisance.

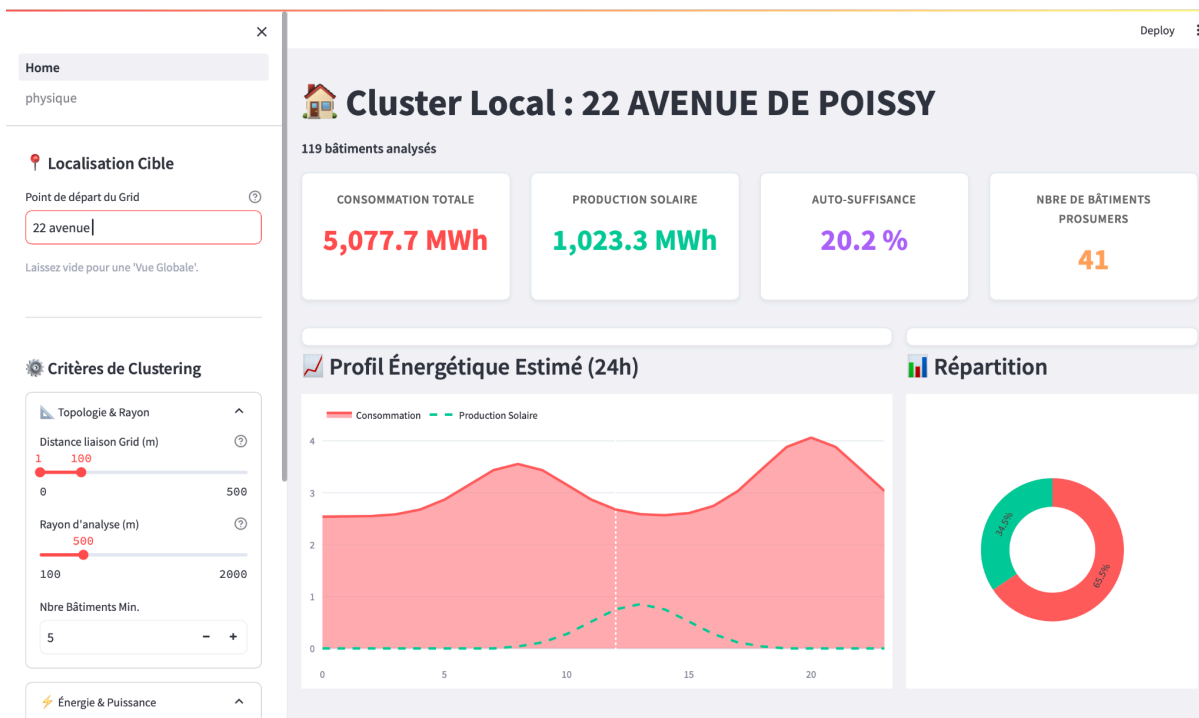
Les cercles colorés autour des points indiquent les zones d'analyse et permettent d'identifier visuellement les clusters énergétiques. Ces éléments facilitent la compréhension de la distribution spatiale de l'énergie et la détection des zones pouvant partager ou échanger de l'électricité.

Aussi, l'utilisateur peut zoomer, se déplacer et survoler les points pour afficher des informations détaillées.

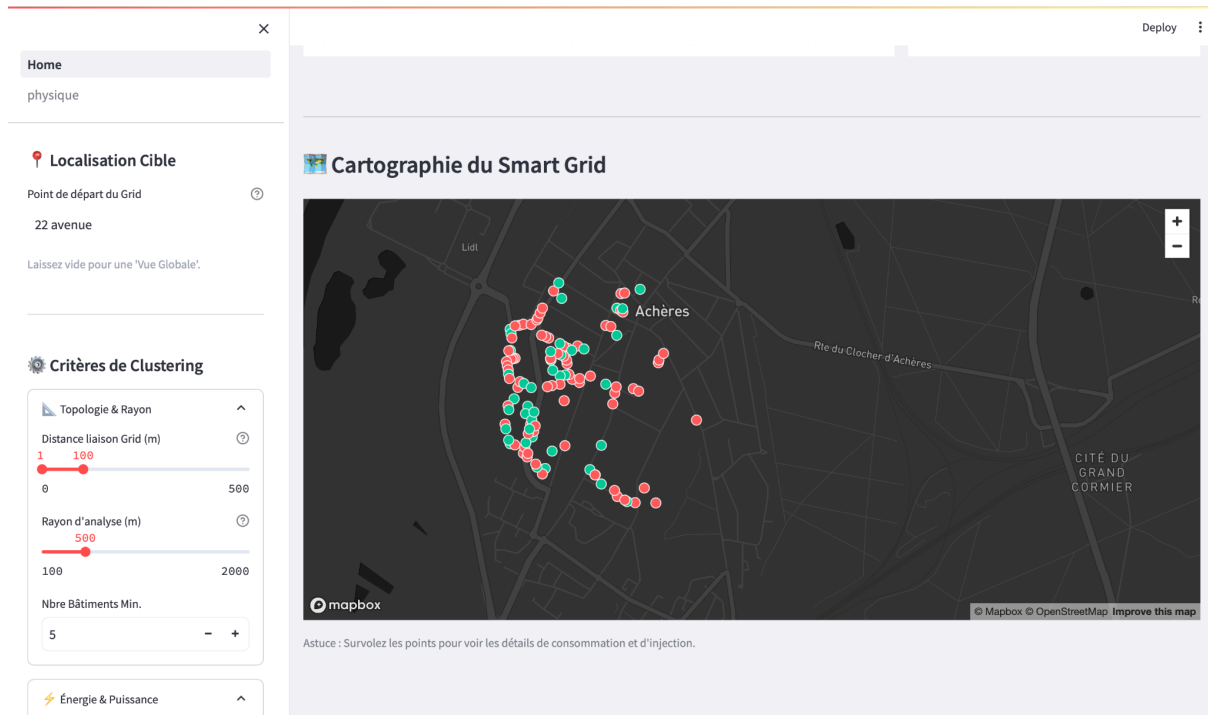


b) Analyse détaillé (Local)

C'est une visualisation détaillée par bâtiment, montrant la consommation et la production solaire autour d'un point d'intérêt.



La figure ci-dessous montre la visualisation des bâtiments et du réseau autour d'une localisation cible. Nous définissons les critères de sélection, tels que l'adresse, le rayon d'analyse, la distance de liaison au réseau et le nombre minimum de bâtiments à inclure dans le cluster. Les bâtiments sont représentés par des points colorés indiquant leur consommation (rouge) et leur production (vert) d'énergie, et il est possible de survoler chaque point pour consulter les détails.



10. Stack utilisé

Pour le développement de notre application, nous avons utilisé un stack adapté aux données spatiales et énergétiques :

- **Frontend** : Streamlit, pour créer une interface interactive et intuitive.
- **Base de données** : Neo4j, une base orientée graphes, permettant de modéliser les relations entre bâtiments, segments de réseau et production solaire.
- **Cartographie** : Pydeck avec Mapbox GL, pour visualiser les bâtiments et les réseaux sur des cartes interactives.
- **Visualisation** : Plotly, pour représenter les courbes de consommation, de production et les KPIs.
- **Langage** : Python 3.10+, offrant flexibilité et compatibilité avec les bibliothèques de data science et de traitement spatial.
- **Framework Big data**: Spark (pyspark) pour traitement de volume important de données

Conclusion

Dans le cadre du projet Smart-Grid, nous avons :

- Intégrer et prétraiter les données RTE, les informations sur les bâtiments et la topologie du réseau basse tension.
- Modéliser les relations entre bâtiments et réseau dans Neo4j sous forme de graphes.
- Simuler la production solaire et générer des courbes de charge journalières pour évaluer l'autosuffisance énergétique et identifier les clusters performants.
- Développer une interface interactive avec visualisations cartographiques et graphiques pour explorer les données et analyser les profils énergétiques.

Annexe

1) Clustering Global

Cette requête **identifie des clusters de bâtiments connectés au même segment de réseau**, calcule leur **consommation totale et leur production solaire potentielle**, puis **sélectionne ceux dont le taux d'autosuffisance dépasse un seuil défini**.

Elle retourne ensuite **les indicateurs énergétiques (production, consommation, autosuffisance, nombre de bâtiments)** pour chaque cluster et pour l'ensemble du territoire.

```
// 1. Récupération des bâtiments et de leurs segments de réseau
MATCH (g:GridSegment)-[r:CONNECTED]-(b:Building)
WHERE b.latitude IS NOT NULL AND b.longitude IS NOT NULL
AND r.distance >= $dist_min AND r.distance <= $dist_max
AND b.conso_totale >= $min_conso

// 2. Récupération de l'énergie potentielle (Solaire)
OPTIONAL MATCH (b)-[:HAS_POTENTIAL]->(p:Potentiel)
WITH g, b, coalesce(p.prod_annuelle_mwh, 0) as prod_val

// 3. Agrégation par Segment (Le Cluster)
// On calcule les totaux directement ici pour définir la performance du cluster
WITH g,
count(b) as nb_b,
sum(b.conso_totale) as g_conso,
sum(prod_val) as g_prod,
// Identification des consommateurs purs (production négligeable)
sum(CASE WHEN prod_val < 0.1 THEN 1 ELSE 0 END) as nb_cons,
collect({
  adresse: b.adresse,
  lat: b.latitude,
  lon: b.longitude,
  conso: round(b.conso_totale, 2),
  prod: round(prod_val, 2),
  type: labels(b)
}) as batiments

// 4. Filtrage basé sur les performances GLOBALES du cluster
// On calcule le ratio sur les sommes (g_prod / g_conso)
WITH *,
CASE WHEN g_conso > 0 THEN (g_prod / g_conso) * 100 ELSE 0 END as cluster_ratio
WHERE nb_cons >= 1 // Au moins un consommateur dans le groupe
AND cluster_ratio >= $min_autosuff // Le cluster doit atteindre le seuil global
AND nb_b >= $min_buildings // Taille minimale du cluster

// 5. Calcul des KPIs pour l'ensemble de la sélection territoriale
WITH collect({
  g: g,
  g_conso: g_conso,
  g_prod: g_prod,
  nb_b: nb_b,
  nb_cons: nb_cons,
  ratio: cluster_ratio,
  batiments: batiments
}) as all_clusters

// Réduction pour obtenir les totaux de tous les clusters validés
WITH all_clusters,
reduce(s = 0.0, c IN all_clusters | s + c.g_conso) as total_conso_territoire,
reduce(s = 0.0, c IN all_clusters | s + c.g_prod) as total_prod_territoire,
reduce(s = 0, c IN all_clusters | s + c.nb_b) as total_nb_batiments

// 6. Construction du JSON final
```

```

RETURN {
  kpi_global_cluster: {
    total_conso: round(total_conso_territoire, 2),
    total_prod: round(total_prod_territoire, 2),
    nb_batiments: total_nb_batiments,
    autosuffisance: CASE WHEN total_conso_territoire > 0
      THEN round((total_prod_territoire / total_conso_territoire) * 100, 2)
      ELSE 0 END
  },
  centre: { adresse: "Vue Globale", lon: null, lat: null },
  clusters: apoc.map.fromPairs([item IN all_clusters | [
    "cluster_" + replace(item.g.nom_grd, " ", "_") + "_" + elementId(item.g),
    {
      Kpi: {
        total_prod: round(item.g_prod, 2),
        total_conso: round(item.g_conso, 2),
        autosuffisance: round(item.ratio, 2),
        nb_batiments: item.nb_b,
        nb_consumers: item.nb_cons,
        nom: item.g.nom_grd,
        id_segment: elementId(item.g)
      },
      liste_batiments: item.batiments
    }
  ]])
} as Resultat

```

2) Clustering Local

Cette requête **identifie un cluster de bâtiments autour d'un bâtiment central dans un rayon donné**, puis calcule **leur consommation totale, leur production solaire potentielle et leur autosuffisance énergétique collective**.

Elle retourne **les indicateurs du cluster et la liste détaillée des bâtiments**, uniquement si le groupe **atteint un seuil minimal d'autosuffisance et contient au moins un consommateur pur**.

```

// 1. Trouver le bâtiment servant de centre
MATCH (found_center:Building)
WHERE (toLower(found_center.adresse) CONTAINS toLower($search_term) OR found_center.code_commune = $search_term)
WITH found_center LIMIT 1

// 2. Identifier tous les bâtiments dans le rayon défini
MATCH (b:Building)
WHERE point.distance(point({latitude: b.latitude, longitude: b.longitude}),
  point({latitude: found_center.latitude, longitude: found_center.longitude})) < $radius

// 3. Filtrer par distance au réseau et consommation minimale
MATCH (b)-[r:CONNECTED]->(g:GridSegment)
WHERE r.distance >= $dist_min AND r.distance <= $dist_max
AND b.conso_totale >= $min_conso

OPTIONAL MATCH (b)-[HAS_POTENTIAL]->(p:Potentiel)
OPTIONAL MATCH (b)-[HAS_PROFIL]->(pr:Profil)

WITH b, r, found_center,
  coalesce(p.prod_annuelle_mwh, 0) as prod_val,
  pr, p

// 4. Calcul de l'autosuffisance individuelle (pour info dans la liste)
WITH *, CASE WHEN b.conso_totale > 0 THEN (prod_val / b.conso_totale) * 100 ELSE 0 END as current_asuff

// 5. Agrégation temporaire pour calculer les KPIs totaux du cluster
WITH found_center,
  sum(b.conso_totale) as cluster_total_conso,
  sum(prod_val) as cluster_total_prod,
  count(b) as nb_b,
  collect({

```

```

b: b,
r: r,
pr: pr,
p: p,
prod_val: prod_val,
asuff: current_asuff
}) as cluster_items

// 6. FILTRE CRITIQUE : Autosuffisance GLOBALE et Mixité
// On calcule le ratio sur le total du groupe
WITH *,
CASE WHEN cluster_total_conso > 0
THEN (cluster_total_prod / cluster_total_conso) * 100
ELSE 0 END as cluster_autosuff_ratio
WHERE cluster_autosuff_ratio >= $min_autosuff
AND nb_b >= $min_buildings
// On vérifie qu'il y a au moins un consommateur pur dans les items collectés
AND size([x IN cluster_items WHERE x.prod_val < 0.1]) >= 1

// 7. Reconstruction finale du résultat
RETURN {
kpi: {
total_conso: cluster_total_conso,
total_prod: cluster_total_prod,
nb_batiments: nb_b,
nb_prosumers: size([x IN cluster_items WHERE x.prod_val > 0.1]),
autosuffisance: cluster_autosuff_ratio
},
liste_batiments: [x IN cluster_items | {
uid: x.b.uid,
adresse: x.b.adresse,
lat: x.b.latitude,
lon: x.b.longitude,
type: labels(x.b),
conso: x.b.conso_totale,
prod: x.prod_val,
autosuffisance: x.asuff,
dist_grid: toFloat(x.r.distance),
load_curve_json: x.pr.load_curve,
prod_curve_json: x.p.courbe_charge
}],
centre: {
lat: found_center.latitude,
lon: found_center.longitude,
adresse: found_center.adresse
}
} as Resultat

```